

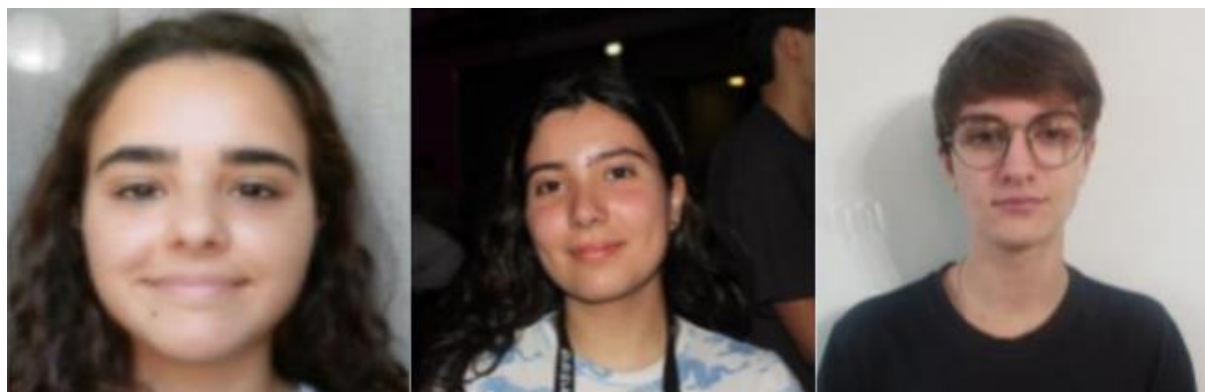
Universidade do Minho



COMPUTACAO GRAFICA

Grupo 16

Fase 2



Trabalho realizado por:
Eduarda Mafalda Martins Vieira, a104098

Maria Leonor Carvalho da Cunha, a103997

Pedro Manuel Macedo Rebelo, a104091

Março 2026

Conteúdo

1. Introdução3
2. Alterações da Fase 1 para a Fase 23
 - 2.1 Estruturas de dados3
 - 2.1.1 TransformType e Transformation3
 - 2.1.2 Group3
 - 2.2 Parser XML4
 - 2.2.1 Parsing de Transformações4
 - 2.2.2 Parsing Recursivo de Grupos4
 - 2.3 Engine - Renderização hierárquica4
 - 2.3.1 Mecanismo de Pilha de Matrizes4
 - 2.3.2 Função renderGroup4
 - 2.3.3 Aplicação de Transformações4
 - 2.3.4 Cache de Modelos4
 - 2.3.5 Resolução de Caminhos5
3. Estrutura e parsing do ficheiro XML5
 - 3.1 Estrutura do ficheiro5
 - 3.2 Parsing do ficheiro5
4. Demonstração - Sistema Solar5
 - 4.1 Exemplo de Herança de Transformações6
6. Decisões de Implementação7
7. Conclusão7

1. Introducao

A Fase 2 do trabalho prático de Computação Gráfica tem como objetivo estender o motor 3D desenvolvido na Fase 1, acrescentando suporte a transformações geométricas hierárquicas. Enquanto na fase anterior apenas era possível carregar modelos numa lista plana, nesta fase a cena passa a ser organizada como uma árvore de grupos, onde cada nó pode conter transformações (translação, rotação e escala) e subgrupos filhos que herdam as transformações dos seus ancestrais.

Esta fase incide exclusivamente sobre a componente do Engine – o gerador de primitivas permanece inalterado. A principal alteração do ponto de vista técnico consiste na implementação de um mecanismo de pilha de matrizes (glPushMatrix/glPopMatrix) que permite isolar corretamente as transformações de cada ramo da hierarquia.

Como cena de demonstração obrigatória foi desenvolvido um modelo estático do sistema solar, com o Sol no centro e planetas a diferentes distâncias, cada um podendo ter os seus próprios satélites definidos como subgrupos filhos.

2. Alteracoes da Fase 1 para a Fase 2

A principal evolução relativamente à Fase 1 encontra-se nas estruturas de dados utilizadas para representar a cena. Foram criados três novos tipos: Transformation, TransformType e um Group reformulado.

2.1 Estruturas de dados

Fase 1: não havia estrutura hierarquica explicita no modelo de cena.

Fase 2: introdução de Group com children para representar a árvore de cena.

Fase 2: introdução de TransformType e Transformation para translate/rotate/scale.

A opcao por vector<Transformation> ordenado preserva a ordem do XML, requisito essencial porque a composicao de transformacoes nao e comutativa.

2.1.1 TransformType e Transformation

O enum TransformType distingue os três tipos de transformação geométrica suportados:

```
enum TransformType { TRANSLATE, ROTATE, SCALE };
```

A struct Transformation armazena um único elemento de transformação juntamente com o seu tipo e parâmetros. O campo angle é utilizado exclusivamente nas rotações; para translações e escalas assume o valor zero.

A decisão de usar uma lista ordenada de Transformation em vez de campos separados por tipo é fundamental: a ordem das transformações é semanticamente relevante em OpenGL, uma vez que as matrizes são aplicadas da direita para a esquerda. Por exemplo, Translate(Rotate(v)) produz um resultado diferente de Rotate(Translate(v)).

2.1.2 Group

O Group foi reformulado para refletir a natureza recursiva de um scene graph. Cada nó contém três coleções:

Os filhos herdam automaticamente as transformações acumuladas do pai através do mecanismo de pilha de matrizes do OpenGL, sem necessidade de qualquer propagação explícita na estrutura de dados.

2.2 Parser XML

O parser foi reestruturado para suportar a nova estrutura hierárquica. A função `parseGroup` é chamada recursivamente para cada elemento `<group>` encontrado, construindo a árvore de grupos de forma natural.

2.2.1 Parsing de Transformações

A função `parseTransforms` itera sobre todos os filhos do elemento `<transform>` pela ordem em que aparecem no XML, utilizando `FirstChildElement` seguido de chamadas sucessivas a `NextSiblingElement`. Esta abordagem garante que a ordem das transformações é preservada exatamente como definida no ficheiro de configuração:

2.2.2 Parsing Recursivo de Grupos

A função `parseGroup` processa um nó `<group>` em três passos sequenciais: primeiro as transformações, depois os modelos e finalmente os subgrupos filhos. Os filhos são encontrados com `FirstChildElement("group")` e depois iterados com `NextSiblingElement("group")`:

2.3 Engine - Renderização hierárquica

Fase 1: renderização sem propagação hierárquica entre grupos.

Fase 2: renderização DFS com `glPushMatrix/glPopMatrix` para herança e isolamento de ramos.

Fase 2: cache de modelos e resolução de caminhos para robustez e eficiência.

2.3.1 Mecanismo de Pilha de Matrizes

O conceito central desta fase é a utilização da pilha de matrizes do OpenGL para isolar as transformações de cada grupo. O OpenGL mantém internamente uma pilha (stack) de matrizes 4x4. As funções `glPushMatrix` e `glPopMatrix` permitem guardar e restaurar o estado atual dessa pilha:

- `glPushMatrix`: copia o topo da pilha e empurra a cópia para cima, ficando disponível para modificação
- `glTranslatef`, `glRotatef`, `glScalef`: multiplicam a matriz do topo pelo operador correspondente
- `glPopMatrix`: descarta o topo da pilha, restaurando o estado que existia antes do push

Este mecanismo garante que as transformações aplicadas a um grupo não afetam os seus grupos irmãos: após o processamento de um filho, o pop restaura o estado que o pai havia guardado.

2.3.2 Função `renderGroup`

A renderização hierárquica é implementada na função `renderGroup`, que segue sempre a mesma sequência para cada nó da árvore:

2.3.3 Aplicação de Transformações

A função `applyTransformation` mapeia cada tipo de transformação para a chamada OpenGL correspondente. O ângulo de rotação é especificado em graus, conforme a convenção do `glRotatef`:

2.3.4 Cache de Modelos

Para evitar carregar o mesmo ficheiro .3d múltiplas vezes (cenários onde o mesmo modelo é referenciado em vários grupos, como acontece no sistema solar onde todas as esferas partilham o mesmo sphere.3d), foi implementada uma cache baseada num map<string, ModelData>. A função collectModelReferences percorre recursivamente a árvore de grupos, recolhe todos os nomes de ficheiros únicos e carrega cada um apenas uma vez antes de inicializar o GLUT.

2.3.5 Resolução de Caminhos

Foi implementada a função resolveModelPath que tenta localizar o ficheiro do modelo em vários locais por ordem de prioridade: no caminho absoluto fornecido, na mesma diretoria do ficheiro XML de configuração, e na diretoria ../generator/files3d/. Esta heurística torna o engine mais robusto a diferentes estruturas de diretoria de projeto.

3. Estrutura e parsing do ficheiro XML

3.1 Estrutura do ficheiro

O ficheiro XML contem um elemento world com window, camera e uma arvore de grupos. Cada group pode conter transform (translate, rotate, scale), models e outros group aninhados. Na implementacao atual, transformacoes sao aplicadas a modelos e subgrupos desse no.

3.2 Parsing do ficheiro

A implementacao atual em parser.cpp lê transformacoes e modelos de cada grupo e processa recursivamente os filhos. A ordem de transformacoes e mantida conforme declarada no XML.

4. Demonstração - Sistema Solar

A cena solar_system.xml demonstra a hierarquia obrigatoria da fase: Sol como referencia, planetas como filhos e luas como filhos dos respectivos planetas. Dois exemplos claros de heranca sao a Lua (filha da Terra) e o anel de Saturno (filho de Saturno).

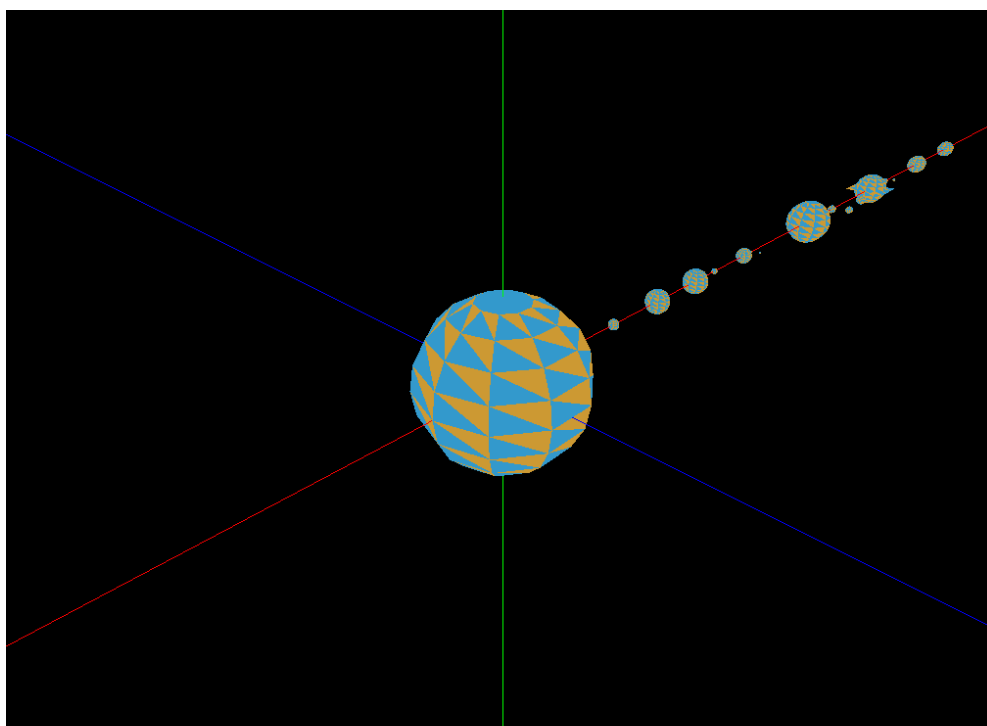


Figura 1 - Vista Geral Superior do Sistema Solar

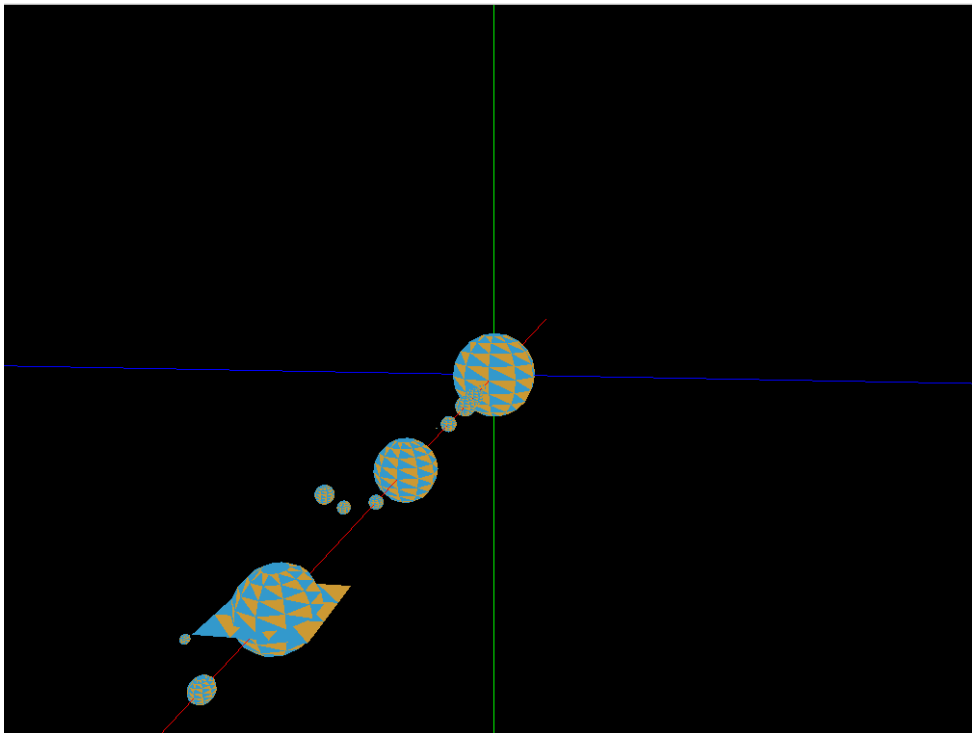


Figura 2 - Vista Geral Perto de Urano

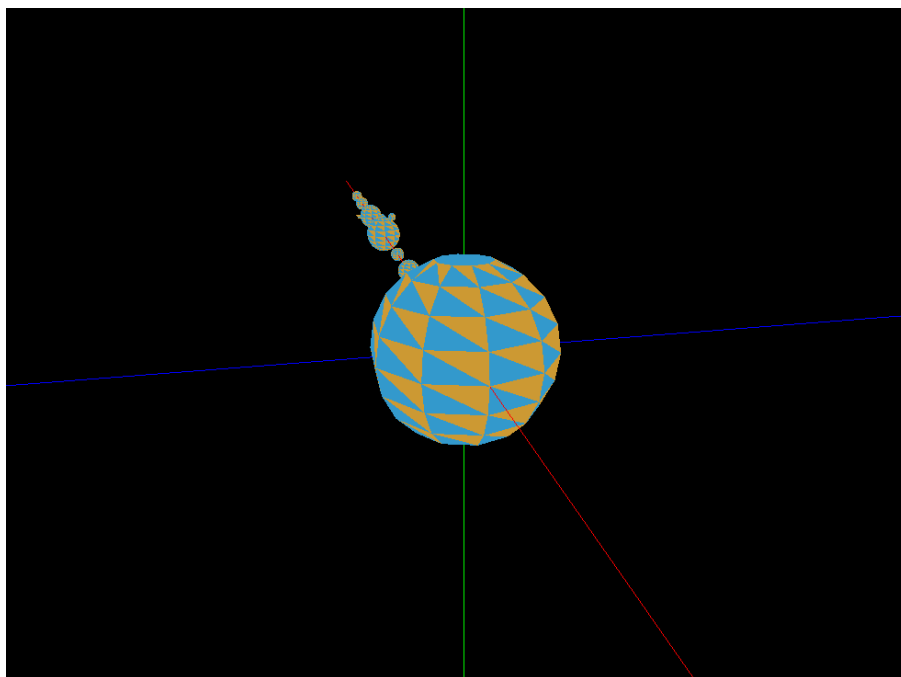


Figura 3 - Sistema Solar numa vista Baixa

4.1 Exemplo de Herança de Transformações

O caso da Lua ilustra de forma clara o funcionamento da herança de transformações. A Lua tem `translate(3, 0, 0)` no seu grupo local. No entanto, como é filha da Terra (que tem `translate(16, 0, 0)` e `scale(0.75, ...)`), a posição final da Lua resulta da composição de todas as transformações ao longo da cadeia hierárquica. O OpenGL realiza esta composição automaticamente através da pilha de matrizes: quando `renderGroup` processa a Lua, a matriz corrente já inclui as transformações da Terra, pelo que a translação da Lua é aplicada no espaço local da Terra.

O anel de Saturno é outro exemplo interessante: é um `plane.3d` (gerado como plano no plano XZ) ao qual se aplica uma escala muito reduzida no eixo Y (0.05) e ampla nos eixos X e Z (3.0), simulando visualmente um anel achatado. Como está num grupo filho de Saturno com `translate(45, 0, 0)`, o anel posiciona-se automaticamente em torno do planeta.

6. Decisões de Implementação

Representar transformações como lista ordenada para garantir semântica do XML.

Usar `Group` recursivo para modelar scene graph.

Aplicar herança via pilha de matrizes em OpenGL (push/pop).

Adicionar cache de modelos para evitar recarregamentos.

Resolver caminhos de modelos com fallback de diretórios.

7. Conclusão

A Fase 2 cumpre a evolução principal do enunciado ao introduzir cenas hierárquicas com transformações geométricas, mantendo a ordem de aplicação e a herança entre nós. O sistema solar valida esse comportamento em casos de planetas, luas e subestruturas como o anel de Saturno.